Full-Stack Project Plan

JWT Authentication — Backend + Frontend

Beginner-Friendly Guide with Production-Ready Notes

SECTION 1: BACKEND

1.1 POST /signup — User Registration

This API endpoint allows new users to create an account by providing their name, email and password.

Request Details

Method	POST
Path	/signup
Body	{ name: "abc", email: "abc@gmail.com", password: "1234" }

Response Codes

500	Server Error — Something went wrong on the server side
409	User Already Exists — Email already registered
201	Success — User created successfully

Why 201 and not 200 for success?

• 200 means 'OK — the request was processed'. It's used for GET requests (fetching data).

• 201 means 'Created — a new resource was created'. Since we're creating a new user, 201 is semantically correct.

Why check for 409 (User Already Exists)?

Email addresses should be unique in a database. If we don't check for duplicates, two users could register with the same email, causing login confusion and security issues.

PRODUCTION-READY NOTES for /signup

- **Hash the password:** Never store passwords as plain text. Use bcrypt: bcrypt.hash(password, 10). This converts '1234' into something like '$2b$10$abc...' which can't be reversed.
- **Validate input:** Use a library like Joi or Zod (shown in Section 3) to ensure name/email/password meet rules before touching the database.
- **Return a JWT token on signup:** Many apps log the user in immediately after signup — return a JWT token so they don't have to login separately.
- **Use HTTPS:** All API communication must happen over HTTPS in production so passwords can't be intercepted.

1.2 POST /login — User Login

This endpoint verifies a user's credentials and, if correct, returns a JWT token that the frontend will use for authenticated requests.

Request Details

Method	POST
Path	/login
Body	{ email: "abc@gmail.com", password: "1234" }

Response Codes

500	Server Error — Internal issue on the backend
401	User Does Not Exist — Email not found in database
201	Success — Login successful, returns JWT token

Why 401 for 'User Does Not Exist'?

401 means 'Unauthorized'. It tells the client: 'You are not allowed because we can't verify who you are.' This is the correct code when credentials don't match.

Note: Some APIs use 404 (Not Found) here, but 401 is better practice because you don't want to reveal whether an email is registered or not (security reason).

What happens inside /login?

- Look up the user in the database by email
- If not found → return 401
- Compare the provided password with the stored hashed password using bcrypt.compare()
- If match → generate a JWT token and return it
- If no match → return 401 (wrong password)

PRODUCTION-READY NOTES for /login

- **Never say which field is wrong:** Don't say 'email not found' or 'wrong password' separately. Just say 'Invalid credentials'. This prevents hackers from confirming which emails are registered.
- **Rate limiting:** Limit login attempts (e.g., max 5 per 10 minutes) to prevent brute-force attacks. Use express-rate-limit library.
- **Return user info with token:** Along with the JWT token, return basic user info (name, email) so the frontend can display it without an extra API call.

1.3 JWT Token + Middleware

After login, the server gives the client a JWT (JSON Web Token). On every protected route, the client must send this token in the request header. The server verifies it using a middleware function.

How the Authorization Header Works

Header Key	authorization
Header Value	jwttoken (the actual token string)

What is a JWT Token?

A JWT is a compact, self-contained string with 3 parts separated by dots:

```
header.payload.signature
```

Example token:

```
eyJhbGciOiJIUzI1NiJ9.eyJlbWFpbCI6ImFiY0BnbWFpbC5jb20ifQ.SIGNATURE
```

- Header: contains the algorithm used (e.g., HS256)
- Payload: contains user data (e.g., email, userId)
- Signature: proves the token wasn't tampered with

How Middleware Intercepts the Token

Middleware is a function that runs BEFORE the actual route handler. It checks if the token is valid:

```
Request → [Middleware: verify token] → Route Handler → Response
```

Protected Route: GET /products

Only authenticated users (those with a valid token) can access this route. It returns:

```
[ { name: "mobile", price: 10000 }, { name: "watch", price: 5000 } ]
```

Server-Side Validation (Joi Library)

The 'J Library' referenced in the plan is Joi — a popular Node.js validation library. It ensures incoming data has the correct shape before you process it.

Example: Ensure email is a valid email and password is at least 6 characters.

PRODUCTION-READY NOTES for JWT + Middleware

- **Token expiry:** Always set an expiry on JWT tokens: jwt.sign(payload, secret, { expiresIn: '1h' }). A token that never expires is a security risk.
- **Refresh tokens:** Use short-lived access tokens (15 min) + long-lived refresh tokens (7 days). When access token expires, use the refresh token to get a new one without re-login.
- **Store secret in environment variable:** Never hardcode the JWT secret. Use process.env.JWT_SECRET loaded from a .env file.
- **Use 'Bearer' prefix:** In production, the standard format is: Authorization: Bearer <token> — middleware should extract the token after 'Bearer '.
- **HTTPS only:** JWT tokens can be stolen if sent over HTTP. Always use HTTPS in production.

SECTION 2: FRONTEND

The frontend is built using React. It connects to the backend APIs built in Section 1. Here's what needs to be implemented:

2.1 Routing & Private Routing

React Router is used to handle navigation between pages (Login, Signup, Home).

What is a Private Route?

A Private Route is a route that only logged-in users can access. If someone tries to visit /home without being logged in, they should be redirected to /login.

How it works

- Check if a JWT token exists (usually in localStorage)
- If token exists → allow access to the page
- If no token → redirect to /login

PRODUCTION-READY NOTES for Routing

- **Validate token on server:** Storing a token in localStorage doesn't mean it's valid. For truly private routes, verify the token with the backend on page load.
- **Use React Router v6:** The latest version has a cleaner API with <Outlet> for nested routes.
- **Handle token expiry:** If the server returns 401 on an API call, automatically redirect to /login and clear the stored token.

2.2 Login Page

The Login page has two responsibilities: validate the form on the client side, then call the /login API and store the returned JWT token.

Client-Side Validation

Before sending the API request, validate the form fields in the browser:

- Email field: check it's not empty and has valid email format
- Password field: check it's not empty

This gives users instant feedback without a network call.

Login API Integration

- On form submit, call POST /login with email and password
- On success (201): extract the JWT token from the response
- Store the token: localStorage.setItem('token', jwtToken)
- Also store user info (name/email) so Home page can display it
- Redirect the user to the Home page
- On failure (401): show error message 'Invalid credentials'

PRODUCTION-READY NOTES for Login Page

- **Use React Hook Form + Zod/Yup:** For robust client-side validation with less boilerplate. Handle loading states and disable the button while the API call is in progress.
- **Consider httpOnly cookies:** Instead of localStorage, store JWT in httpOnly cookies set by the server. This protects against XSS attacks — JavaScript can't read httpOnly cookies.
- **Show loading indicator:** Disable the submit button and show a spinner while awaiting the API response.

2.3 Signup Page

Similar to Login but collects name, email, and password to create a new account.

Client-Side Validation

- Name: not empty, minimum 2 characters
- Email: valid email format
- Password: minimum 6 characters
- Optional: Confirm password field that must match password

Signup API Integration

- On form submit, call POST /signup with name, email, password
- On success (201): either redirect to /login, or auto-login by storing the returned token
- On failure (409): show error 'Email already registered'
- On failure (500): show generic error 'Something went wrong, please try again'

PRODUCTION-READY NOTES for Signup Page

- **Password strength indicator:** Show users whether their password is weak/medium/strong in real time.
- **Email verification:** In production, send a verification email after signup. Don't allow login until email is verified.
- **Auto-login after signup:** Better UX: after successful signup, automatically log the user in (store the token and redirect to Home).

2.4 Home Page

The Home page is only accessible to logged-in users. It demonstrates how to use the stored JWT token for authenticated API calls.

Features to Implement

- Show the logged-in username (retrieved from localStorage or from the token)
- Show a Logout button that clears the token and redirects to /login
- Call GET /products with the JWT token in the header
- Display the list of products returned by the API

How to Attach JWT Token to API Calls

When calling /products, include the token in the Authorization header:

```
headers: { authorization: localStorage.getItem('token') }
```

What 'logout' means

Logout simply removes the token from localStorage. The server doesn't need to be notified (in a basic implementation). Once the token is removed, the private route will redirect to /login.

PRODUCTION-READY NOTES for Home Page

- **Use Axios interceptors:** Instead of manually adding headers to every request, configure Axios to automatically attach the JWT token to all outgoing requests.
- **Handle expired tokens:** If /products returns 401, it means the token has expired. Automatically redirect to /login and clear the stored token.
- **Token invalidation on logout:** In production, maintain a token blacklist on the server, or use short-lived tokens so logout is effective immediately.

SECTION 3: QUICK REFERENCE

HTTP Status Codes Used in This Project

201	Created — a new resource was successfully created
401	Unauthorized — credentials missing or invalid
409	Conflict — resource already exists (e.g., duplicate email)
500	Internal Server Error — unexpected server-side error

Technology Stack

Backend	Node.js + Express + MongoDB (or any SQL DB)
Auth	JWT (jsonwebtoken library) + bcrypt for password hashing
Validation	Joi or Zod (server-side) + React Hook Form (client-side)
Frontend	React + React Router v6 + Axios for API calls
Token Storage	localStorage (learning) → httpOnly cookies (production)

Complete Flow Summary

1. User visits /signup → fills form → frontend validates → calls POST /signup → account created
2. User visits /login → fills form → frontend validates → calls POST /login → server returns JWT token
3. Frontend stores JWT token in localStorage
4. User visits /home (private route) → token found → page loads
5. /home calls GET /products with token in header → middleware verifies token → products returned

6. User clicks Logout → token cleared → redirected to /login

Good luck with your project! Build the basic version first, then add production features one by one.